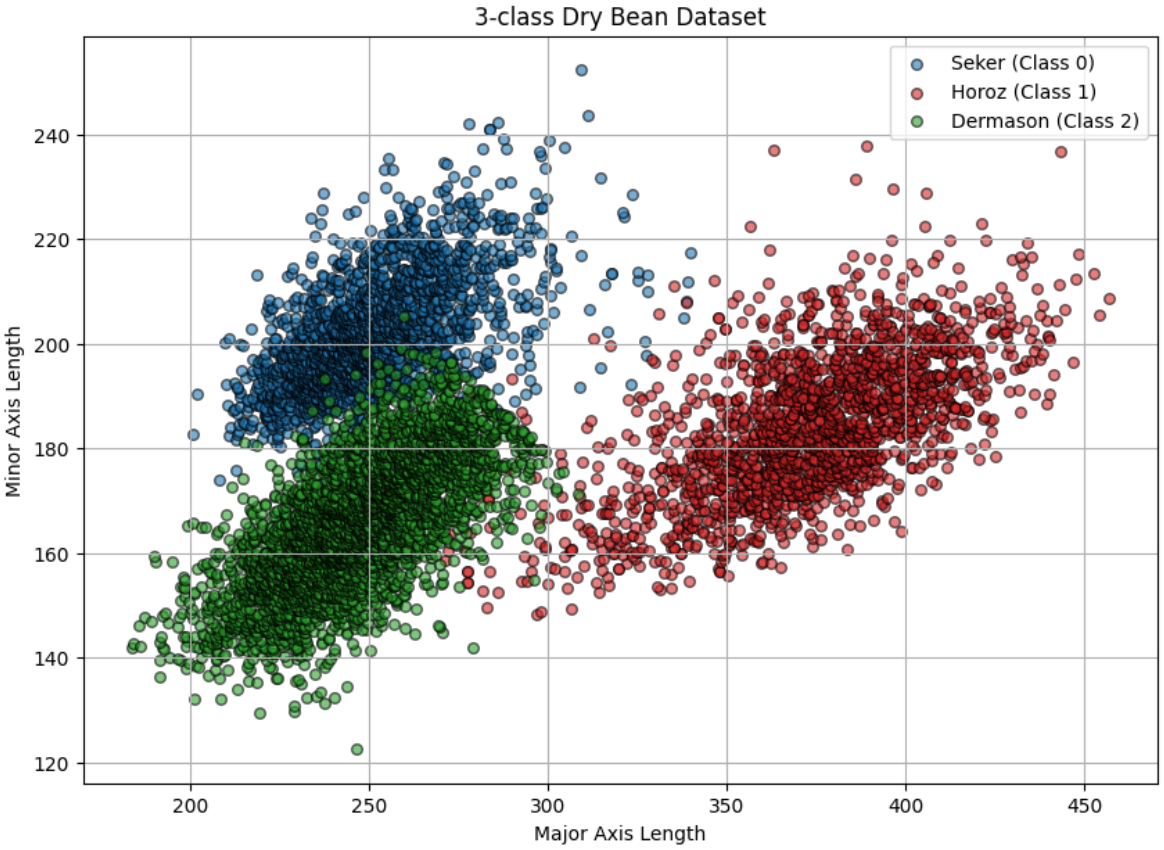
ML ORANGE Problem

Name : Duggireddy Hitesh Pranav Reddy SRN : PES1UG23EC912

In [5]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier
from matplotlib.colors import ListedColormap
from sklearn.metrics import accuracy_score

# Load dataset
df = pd.read_excel('Dry_Bean_Dataset.xlsx')

print(f"Original dataset size: {len(df)} rows")

# Clean labels
df['Class'] = df['Class'].astype(str).str.strip().str.upper()

# Filter 3 classes
df = df[df['Class'].isin(['SEKER', 'HOROZ', 'DERMASON'])]

print(f"Dataset size after filtering to 3 classes: {len(df)} rows")

# Encode classes
class_map = {'SEKER': 0, 'HOROZ': 1, 'DERMASON': 2}
df['class_encoded'] = df['Class'].map(class_map)

# Features
X = df[['MajorAxisLength', 'MinorAxisLength']].values
y = df['class_encoded'].values

# Plot dataset
plt.figure(figsize=(10, 7))

classes = ['Seker (Class 0)', 'Horoz (Class 1)', 'Dermason (Class 2)']
colors = ['#1f77b4', '#d62728', '#2ca02c']

for i in range(3):
    plt.scatter(
        X[y == i, 0], X[y == i, 1],
        c=colors[i], label=classes[i],
        alpha=0.6, edgecolors='k', s=30
    )

plt.title('3-class Dry Bean Dataset')
plt.xlabel('Major Axis Length')
plt.ylabel('Minor Axis Length')
plt.legend()
plt.grid(True)
plt.show()
```

```
Original dataset size: 13611 rows
Dataset size after filtering to 3 classes: 7501 rows
```



Explanation:

This section performs data loading, preprocessing, and initial visualization.

- The dataset is loaded using `pandas.read_excel` .
- The `Class` column is cleaned by:
  - Converting all values to string
  - Removing extra spaces using `strip()`
  - Converting to uppercase using `upper()` This ensures consistent labels and avoids filtering errors.
- The dataset originally contains 7 classes, but it is filtered to only 3 classes:
  - SEKER
  - HOROZ
  - DERMASON
- These classes are encoded into numeric values (0, 1, 2) using a mapping dictionary. This is required because machine learning models work with numerical labels.
- Two features are selected:
  - MajorAxisLength (length of the bean)
  - MinorAxisLength (width of the bean)
- A scatter plot is generated:

  - Each class is shown in a different color
  - This helps visualize how the classes are distributed in 2D space

Output:

- Console prints dataset size before and after filtering
- A scatter plot showing the distribution and overlap of the three classes

In [6]:
```
# Split data
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.15, random_state=42, stratify=y
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=(0.15/0.85), random_state=42, stratify=y_te
)

print(f"Train size: {len(X_train)} // Val size: {len(X_val)} // Test size

# Scaling
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

```
Train size: 5250 // Val size: 1125 // Test size: 1126
```

Explanation:

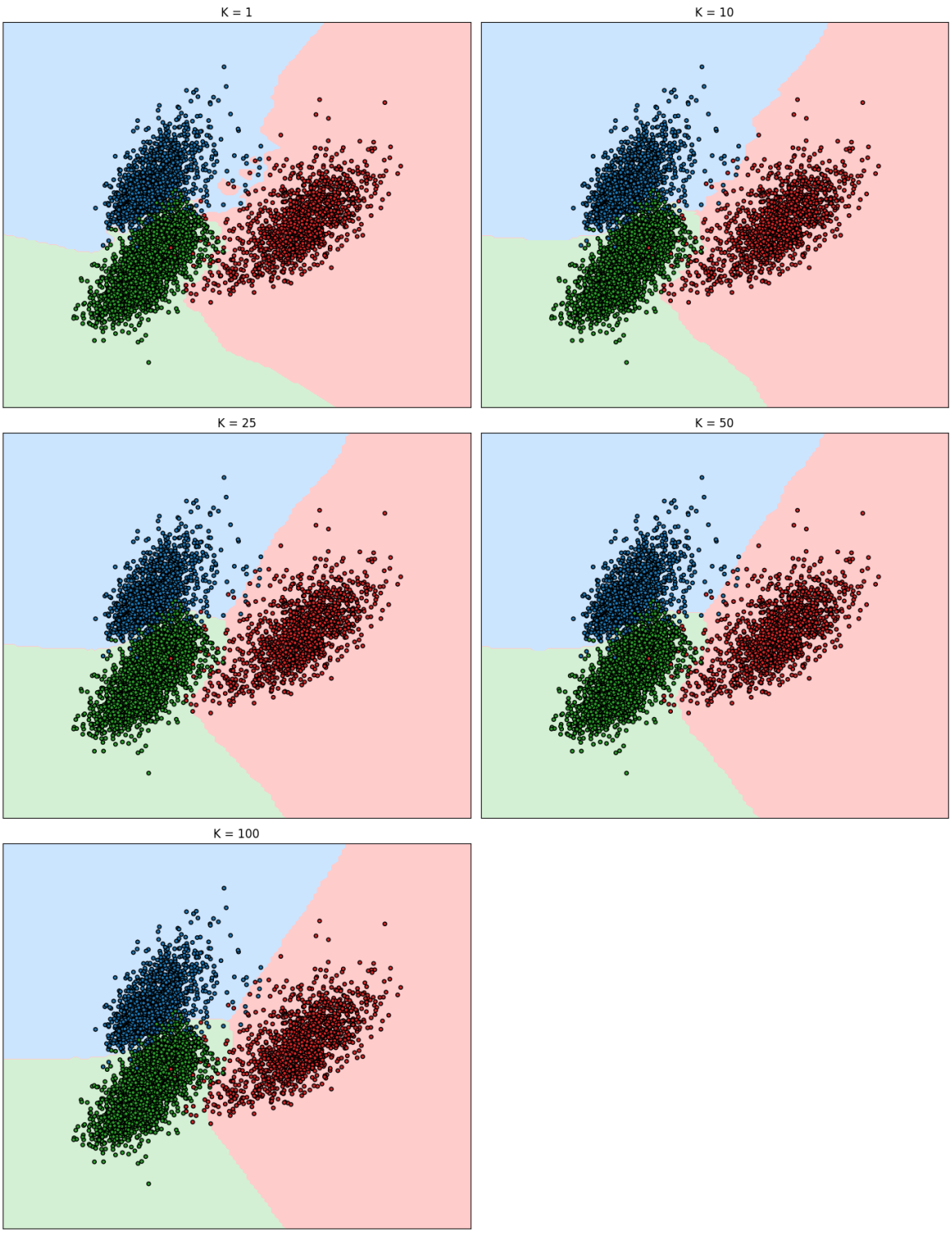
This section prepares the data for training by splitting and scaling.

- The dataset is split into three parts:
  - Training set (70%) → used to train the model
  - Validation set (15%) → used to tune hyperparameters
  - Test set (15%) → used for final evaluation
- The `stratify=y` parameter ensures that all splits maintain the same class distribution.
- Feature scaling is performed using `StandardScaler`:
  - Training data is scaled using `fit_transform()`
  - Validation and test data are scaled using `transform()`
- Scaling is crucial for KNN because:
  - KNN uses Euclidean distance
  - Features with larger values would otherwise dominate the distance calculation

Output:

- Console output showing sizes of train, validation, and test sets

In [7]:
```
k_values = [1, 10, 25, 50, 100]

cmap_light = ListedColormap(['#cce5ff', '#ffcccc', '#d4f0d4'])
cmap_bold = ListedColormap(['#1f77b4', '#d62728', '#2ca02c'])

h = 0.05
x_min, x_max = X_train_scaled[:, 0].min() - 1, X_train_scaled[:, 0].max()
y_min, y_max = X_train_scaled[:, 1].min() - 1, X_train_scaled[:, 1].max()

xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

plt.figure(figsize=(14, 18))

for i, k in enumerate(k_values):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    ax = plt.subplot(3, 2, i + 1)
    ax.contourf(xx, yy, Z, cmap=cmap_light)

    ax.scatter(X_train_scaled[:, 0], X_train_scaled[:, 1],
               c=y_train, cmap=cmap_bold, edgecolor='k', s=15)

    ax.set_title(f"K = {k}")
    ax.set_xticks(())
    ax.set_yticks(())

plt.tight_layout()
plt.show()
```



Explanation:

This section visualizes how the KNN model classifies data for different values of K.

- K values used: 1, 10, 25, 50, 100
- A meshgrid is created:
  - This generates a dense grid of points covering the feature space
  - Each point represents a possible input
- For each K:

    - A KNN model is trained on the scaled training data
    - The model predicts the class for every point in the grid
    - Predictions are reshaped and plotted using `contourf`
  - The training data points are overlaid on the decision boundary

  - Observations:

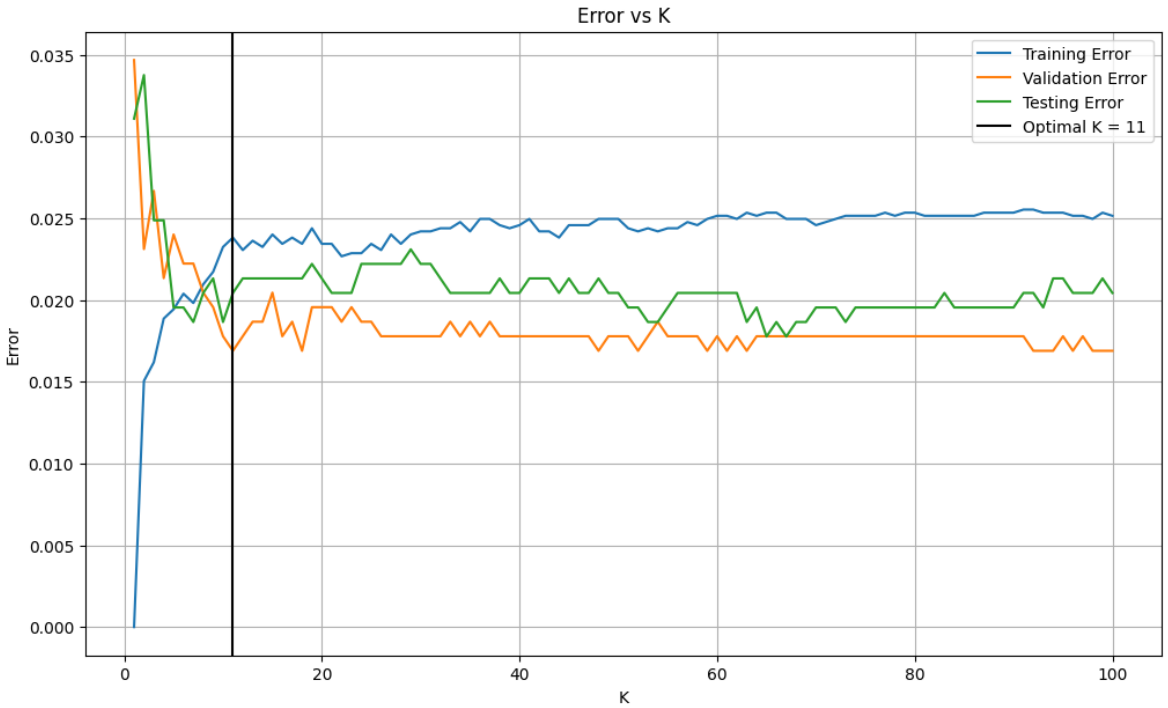
    - Small K (e.g., K=1):
      - Highly irregular boundaries
      - Sensitive to noise → overfitting
    - Large K (e.g., K=100):
      - Very smooth boundaries
      - May ignore important patterns → underfitting

Output:

- Multiple plots showing decision boundaries for different values of K

In [8]:
```
k_sweep = range(1, 101)

train_errors = []
val_errors = []
test_errors = []

for k in k_sweep:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    train_errors.append(1 - accuracy_score(y_train, knn.predict(X_train_s
    val_errors.append(1 - accuracy_score(y_val, knn.predict(X_val_scaled)
    test_errors.append(1 - accuracy_score(y_test, knn.predict(X_test_scal

best_k = k_sweep[np.argmin(val_errors)]

plt.figure(figsize=(12, 7))

plt.plot(k_sweep, train_errors, label='Training Error')
plt.plot(k_sweep, val_errors, label='Validation Error')
plt.plot(k_sweep, test_errors, label='Testing Error')

plt.axvline(x=best_k, color='black', label=f'Optimal K = {best_k}')

plt.xlabel("K")
plt.ylabel("Error")
plt.title("Error vs K")
plt.legend()
plt.grid(True)

print(f"Optimal K: {best_k}")
plt.show()
```

```
Optimal K: 11
```



Explanation:

This section evaluates model performance across different values of K and finds the optimal value.

- K is varied from 1 to 100
- For each K:
  - A KNN model is trained
  - Error rates are computed for:
    - Training set
    - Validation set
    - Test set
- Error is calculated as: Error = 1 - Accuracy
- All errors are stored and plotted against K
- The optimal K is selected as the value that minimizes the validation error:
  - This ensures the model generalizes well to unseen data
- The graph illustrates the bias-variance tradeoff:
  - Small K → low training error, high variance (overfitting)
  - Large K → higher training error, high bias (underfitting)
  - Optimal K → balance between both

Output:

- A graph showing error vs K for training, validation, and test sets
- Printed optimal K value (approximately 11)

Observations:

- The dataset shows partial overlap between the three classes (Seker, Horoz, Dermason), indicating that classification is non-trivial and requires a well-tuned model.
- For very small values of K (e.g., K = 1):
  - The decision boundary is highly irregular and closely follows the training data.
  - The model captures noise and outliers.
  - This results in overfitting (high variance, low bias).
- For moderate values of K:
  - The decision boundary becomes smoother and more generalized.
  - The model captures the underlying data distribution more effectively.
- For very large values of K (e.g., K = 100):
  - The decision boundary becomes overly smooth.
  - The model ignores local patterns and class distinctions.
  - This leads to underfitting (high bias, low variance).
- From the error vs K graph:
  - Training error increases as K increases.
  - Validation error first decreases, reaches a minimum, and then increases.
  - Test error follows a similar trend as validation error.
- The optimal value of K is approximately 11, where validation error is minimum and model generalization is best.

Conclusion:

- The K-Nearest Neighbors (KNN) algorithm was effectively applied to classify the Dry Bean dataset based on morphological features.
- The performance of the KNN model is highly dependent on the choice of the hyperparameter K.
- Small values of K lead to overfitting, while large values of K lead to underfitting. Therefore, selecting an appropriate K is essential for achieving optimal performance.
- Feature scaling plays a crucial role in KNN, as the algorithm relies on distance calculations. Without scaling, features with larger magnitudes would dominate the model.
- By performing a systematic hyperparameter sweep and analyzing error trends, the optimal value of K was determined to be approximately 11.

- This experiment demonstrates the importance of model tuning and validates the bias-variance tradeoff in practical machine learning applications.